

DOMA ASSIST

Assistência por áudio em um relógio Wear OS

Aplicativo acadêmico para comunicação e orientação de funcionários com necessidades especiais, oferecendo informações por voz para pessoas com deficiência visual.

Projeto	Configuração
Plataforma	Wear OS Small Round / Android 11 / API 30
Implementação	Kotlin, Activity nativa, ListView e Button
Funções	Mensagens, notificações, comandos, alertas e instruções por áudio
Entrega	Código, APK debug, testes, capturas e esta documentação
Data	05/10/2026

Repositório público: github.com/Cl4y7oon/doma-assist-wear-os

Aluno: CLAYTON LUIS MEIRELES GONÇALVES

Matrícula: 202501537006

Fonte dos requisitos: Trabalho Prático _ DGT2816 Interação com sensores de smartphones e wearebles.pdf, 22 páginas.

1. Objetivo e requisitos

A empresa fictícia Doma necessita de assistência por áudio: leitura de mensagens e notificações, resposta a comandos, treinamento e alertas de segurança. O aplicativo reúne essas funções em uma interface simples para relógio.

Requisito do roteiro	Realização
Android Studio e emulador	Ambiente instalado; AVD Small Round/API 30 executado.
Projeto Wear OS, mínimo API 30	Módulo app; recurso watch obrigatório; minSdk e targetSdk 30.
Lista, botão e Activity	MainActivity em Kotlin, activity_main.xml com ListView e botões nativos.
BODY_SENSORS / WAKE_LOCK	Declaradas como solicitado. Não há coleta de sinais corporais nem solicitação dessa permissão em execução.
Alto-falante / Bluetooth A2DP	AudioHelper usa FEATURE_AUDIO_OUTPUT e getDevices(GET_DEVICES_OUTPUTS).
Dispositivos conectados/removidos	AudioDeviceCallback permanece registrado na Application e atualiza a interface.
Configuração Bluetooth	Intent ACTION_BLUETOOTH_SETTINGS e extras apresentados no enunciado.
Reprodução de áudio	Text-to-Speech em português, foco de áudio, repetição e interrupção.
Mensagens e notificações	Mensagem demo e NotificationListenerService para texto recebido pelo Android.
Comandos de voz	Reconhecedor do sistema; interpretação do resultado em CommandRouter.
Alertas e instruções	Textos demonstrativos de segurança e treinamento, identificados como exercício.
Git e PDF	Projeto, evidências e documentação publicados no repositório público.

A página 19 permite simulação Wear OS ou dispositivo real. A solução utiliza o emulador. Flutter foi instalado por constar nos materiais; o aplicativo segue as APIs nativas e exemplos Kotlin da prática final. A Activity do tutorial inicial em Java foi adaptada para Kotlin.

2. Organização e funcionamento

Arquivo	Responsabilidade
MainActivity.kt	Lista, botões, telas de conteúdo, configuração Bluetooth e reconhecimento de fala.
AudioHelper.kt	Consulta de saídas e detecção dinâmica de conexão/desconexão.
SpeechPlayer.kt	Inicializa TTS, seleciona português, solicita foco de áudio, fala e repete.
DomaNotificationListener.kt	Recebe notificações, extrai texto e publica a atualização interna.
DomaApplication.kt	Mantém áudio e preferências durante a vida do processo.
CommandRouter.kt	Normaliza acentos/pontuação e encaminha comandos conhecidos.

Fluxo de notificações

Notificação publicada no Android → serviço autorizado → extração de título e texto → armazenamento privado da última mensagem → atualização da interface → leitura automática, se habilitada pelo usuário.

Notificações permanentes, resumos de grupo e notificações sem texto legível são ignoradas. A leitura automática começa pausada. O usuário pode ativá-la, interrompê-la e apagar o último texto. O aplicativo não envia notificações a um servidor nem conserva um histórico completo.

Fluxo de voz

Botão Falar comando → Activity de reconhecimento do sistema → resultado textual → CommandRouter → ação. Não há gravação contínua ou escuta permanente. Se o serviço estiver indisponível ou não retornar uma frase, a interface informa o ocorrido.

Comandos	Ação
ler notificação / repetir	Ler a última notificação ou repetir a última fala.
mensagem / instruções	Ouvir conteúdo demonstrativo e orientações de treinamento.
alerta de segurança	Ouvir um exercício de segurança; não é uma emergência real.
pausar / retomar / Bluetooth	Controlar leitura automática ou abrir a conexão de áudio.

3. Evidências da interface

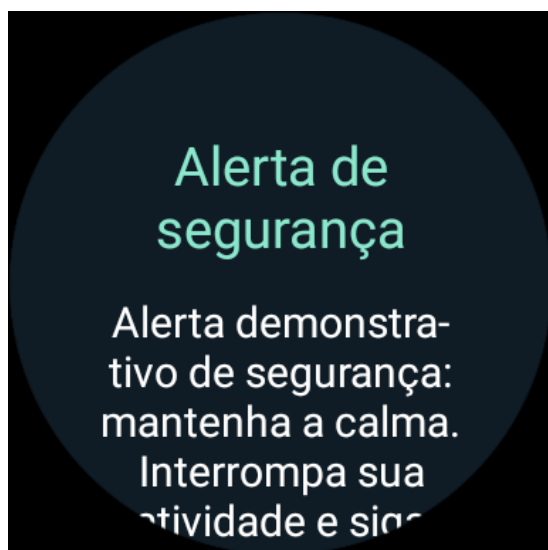
Capturas reais do aplicativo no AVD Wear OS Small Round/API 30, salvas pelo recurso Take Screenshot do Android Studio no painel Running Devices. Cenários preparados pelo modo debug. A origem das imagens é a IDE; não o app complementar descrito na microatividade de apoio.



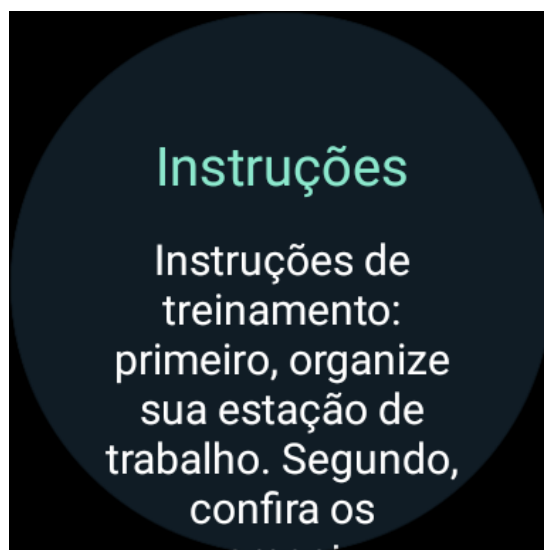
Tela inicial: saída de áudio, botão de voz e lista de opções.



Texto de notificação demonstrativa realmente recebida pelo serviço do sistema.



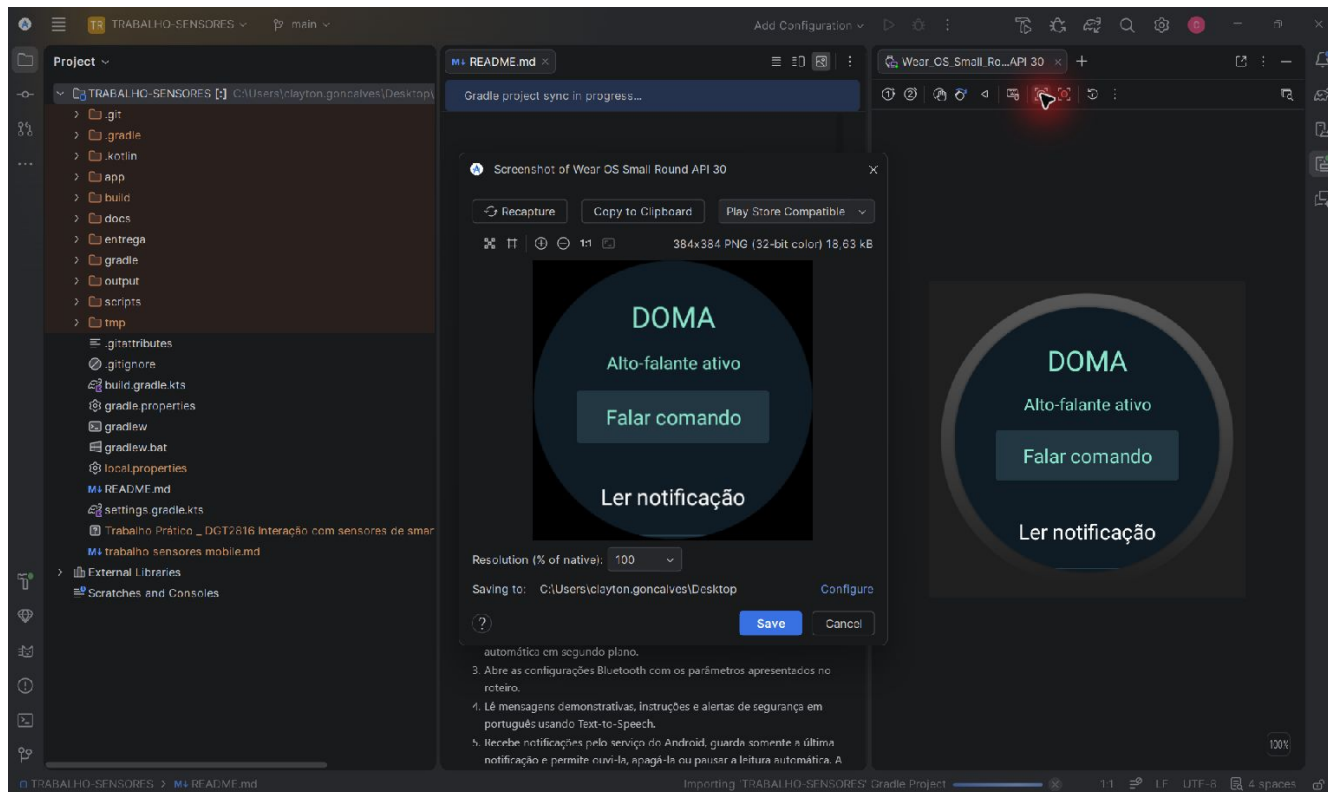
Exercício de segurança; o restante do texto é acessível por rolagem.



Treinamento em texto e TTS; o restante é acessível por rolagem.

3.1. Registro da captura na IDE

Android Studio com o projeto aberto, o relógio no painel Running Devices e o diálogo Screenshot of Wear OS Small Round API 30. A imagem registra o procedimento usado para salvar as quatro capturas da página anterior.



As imagens foram salvas inicialmente na Área de Trabalho pelo botão Save da IDE e copiadas, sem alteração, para docs/capturas/. As capturas anteriores por ADB também foram preservadas, com sua origem identificada no README.

Escopo das microatividades e da entrega

Na página 1, as microatividades são apresentadas como apoio, e a entrega esperada é o Trabalho Prático descrito depois delas, nas páginas 18-22. A microatividade 5 explica outra forma de captura, pelo app complementar no smartphone; esse procedimento não foi executado no ambiente usado.

A captura complementar permanece um objetivo didático citado, sem ser apresentada como realizada. O roteiro final admite simulação e não exige aquisição de hardware nem repete essa captura como arquivo obrigatório de entrega. As evidências deste projeto documentam a alternativa de simulação.

4. Construção e reprodução

Abrir esta pasta no Android Studio, sincronizar o Gradle, selecionar JDK 17 ou 21 para a construção e iniciar o AVD no Device Manager. O runtime da IDE pode ser mais recente do que o runtime escolhido para o Gradle.

Componente	Versão usada
Gradle Wrapper	8.11.1, com verificação SHA-256 da distribuição
Android Gradle Plugin	8.9.2
Kotlin	2.1.20
SDK de compilação / Build Tools	API 35 / 35.0.1
Android mínimo e alvo	API 30 / API 30
Dispositivo de teste	Wear OS Small Round, imagem Wear OS 3 x86 / API 30

Comandos de construção e teste

No PowerShell, executar **.\scripts\executar.ps1** para compilar, instalar e abrir. Executar **.\scripts\testar.ps1** para testes unitários, Lint e testes instrumentados. O roteiro de testes é restrito a emuladores Wear OS e habilita acesso às notificações para a demonstração.

Autorização e compatibilidade

A imagem API 30 testada não possui Activity para a autorização de notificações. No AVD, conceder acesso explicitamente com:

```
adb -s emulator-5554 shell cmd notification allow_listener  
br.com.doma.assist/br.com.doma.assist.DomaNotificationListener
```

O aplicativo não executa essa concessão. Em dispositivos que oferecem a tela, o botão Autorizar notificações abre a configuração do sistema após explicar o acesso. Em um relógio físico, verificar a disponibilidade da autorização e do serviço de reconhecimento.

Demonstração no emulador

Para repetir os cenários, abra Teste por texto (debug) no menu, digite um comando ou publique Notificação demo. As ações permitem verificar mensagens, notificações, instruções e alertas com TTS. Para registrar a tela, utilize Running Devices > Take Screenshot. O modo debug valida transcrições e não representa captação acústica.

5. Resultados, limites e entrega

Verificação	Resultado comprovado
APK debug e Android Lint	Compilação aprovada; 0 erros de Lint.
Testes de comandos	3 testes unitários aprovados: normalização, desconhecidos e pausa/retomada.
Testes no relógio	3 testes instrumentados aprovados: hardware, extração de texto e listener real.
TTS em português	Mecanismo conectado e callbacks START/DONE registrados.
Pausa e retomada	Estados de leitura automática conferidos no emulador.
Interface	Execução e capturas de início, notificação, alerta, instruções e mensagem.

Alcance da validação

A integração com o reconhecedor do sistema e os comandos está implementada; os testes validam transcrições, sem comprovar captação acústica. Detecção A2DP e callback estão implementados, sem teste de fone físico. Não houve avaliação humana do som ou captura pelo app complementar. Esses são limites das evidências, sem exigência de aquisição de dispositivos para a alternativa de simulação prevista na página 19.

O modo de simulação é limitado a builds debug. Alertas e instruções são exemplos acadêmicos; não há integração com sistemas reais de emergência. As limitações estão registradas para que a evidência entregue não seja confundida com validação em hardware real.

Arquivos e envio

A página 22 pede projeto e documentação PDF no Git, ambos publicados no repositório público indicado na capa. Código em app/; PDF em output/pdf/; capturas em docs/capturas/; APK de apoio em entrega/; relatórios, rastreabilidade e revisão em docs/. A entrega é concluída com o envio do link ao tutor pela Sala de Aula Virtual, aba Trabalhos, dentro do prazo.

O enunciado não fornece modelo de relatório, padrão ABNT ou número de páginas. Este PDF organiza identificação, objetivo, implementação, evidências, reprodução e resultados. APK e relatórios de testes são complementos; não há exigência de loja de aplicativos ou vídeo.

Referências

[Áudio em Wear OS](#)

[Entrada de voz](#)

[NotificationListenerService](#)

[Android Gradle Plugin 8.9](#)